CAMPUS RECRUITMENT TRAINING & PLACEMENT BOARD

OFFICIAL PRESCRIBED PLACEMENT MASTER TEXTBOOK

Data Engineering & Big Data Systems (Prescribed Textbook)

Subject Track: Data Engineering & Big Data

Edition: 2026 Campus Placement Edition

SYLLABUS & CURRICULUM OVERVIEW

Comprehensive campus placement textbook covering distributed computing (Hadoop/Spark), data lakehouses (Delta/Parquet), stream processing (Kafka), workflow orchestration (Airflow), NoSQL, database sharding, and architecture case studies.

STRICT EXAMINATION GROUNDING GUARANTEE

All campus placement assessments, live code challenges, milestone tests, and company interview drills for Data Engineering & Big Data on this platform are derived strictly and exclusively from the theoretical concepts, algorithms, worked examples, and placement problems documented in this official textbook. Mastery of these chapters guarantees 100% exam preparation.

PRESCRIBED CHAPTER SYLLABUS (8 CHAPTERS):

Ch 1: Big Data Fundamentals & Distributed Computing Paradigm

Ch 2: Apache Spark Architecture & Distributed Processing

Ch 3: Data Warehousing vs Data Lakes vs Lakehouses

Ch 4: Distributed Messaging & Streaming Architecture

Ch 5: ETL / ELT Pipeline Architecture & Data Orchestration

Ch 6: NoSQL Databases: Document, Key-Value, Columnar & Graph

Ch 7: Data Modeling, Partitioning & Sharding at Scale

Ch 8: Solved Data Engineering Placement Case Studies & System Interview

Author: Prescribed Courseware Architectural Board

Verified Complete Unabridged Placement Master Edition | Internal Campus Circulation

Edition: 2026 Campus Placement Edition

Subject: Data Engineering & Big Data

Courseware Board: Prescribed Placement Architectural Board

Status: In-App Complete Master Textbook (Full Curriculum Available In-Platform)

> **? EXAM GROUNDING POLICY:** All campus placement technical assessments, coding lab challenges, and technical interview evaluations are strictly grounded in the concepts, algorithms, formulas, step-by-step solved examples, practice questions, and interview questions detailed in this textbook.

· Course Overview & Curriculum Scope

Comprehensive campus placement textbook covering distributed computing (Hadoop/Spark), data lakehouses (Delta/Parquet), stream processing (Kafka), workflow orchestration (Airflow), NoSQL, database sharding, and architecture case studies.

· Complete Table of Contents (Chapter-by-Chapter)

- [Chapter 1: Big Data Fundamentals & Distributed Computing Paradigm](#chapter-1)
- [Chapter 2: Apache Spark Architecture & Distributed Processing](#chapter-2)
- [Chapter 3: Data Warehousing vs Data Lakes vs Lakehouses](#chapter-3)
- [Chapter 4: Distributed Messaging & Streaming Architecture](#chapter-4)
- [Chapter 5: ETL / ELT Pipeline Architecture & Data Orchestration](#chapter-5)
- [Chapter 6: NoSQL Databases: Document, Key-Value, Columnar & Graph](#chapter-6)
- [Chapter 7: Data Modeling, Partitioning & Sharding at Scale](#chapter-7)
- [Chapter 8: Solved Data Engineering Placement Case Studies & System Interview Drills](#chapter-8)

CHAPTER 1

Big Data Fundamentals & Distributed Computing Paradigm

Key Principle / Exam Takeaway: Master the 5 V's of Big Data, Hadoop Ecosystem, HDFS NameNode/DataNode architecture, replication, and MapReduce execution model.

Core Curriculum Topics Covered in this Chapter:

- The 5 V's: Volume, Velocity, Variety, Veracity, Value
- Hadoop Distributed File System (HDFS): NameNode (metadata), DataNode (block storage), 128MB block sizing
- HDFS Replication factor (default 3), rack awareness, and heartbeats
- MapReduce execution lifecycle: Map, Shuffle and Sort, Reduce
- YARN (Yet Another Resource Negotiator): ResourceManager and NodeManager

· 1. Core Concepts & In-Depth Explanation

Big Data describes datasets whose scale exceeds traditional single-node relational databases:

- **HDFS Architecture:** Master-Worker architecture. The **NameNode** stores filesystem metadata in RAM for microsecond lookup; **DataNodes** store raw block data (default 128MB chunks) on local drives. Default replication factor is 3: block 1 is stored on local rack DataNode A, replica 2 on same rack DataNode B, replica 3 on remote rack DataNode C (**Rack Awareness**).
- **MapReduce:** Functional distributed computing model:

1. `Map`: Emits key-value pairs `(k1, v1) -> list(k2, v2)`. 2. `Shuffle & Sort`: The network-heavy barrier where framework groups all values matching identical intermediate keys to the same worker node. 3. `Reduce`: Aggregates grouped values `(k2, list(v2)) -> list(k3, v3)`.

· 2. Key Formulas, Rules & Complexity Classes

1. **HDFS Block Sizing Rationale:** $$\text{128MB Block Size} \implies \text{Minimizes disk seek time to } < 1\% \text{ of data transfer time}$$ 2. **Rack Awareness Strategy:** $$\text{Replica 1 (Local Node)} \to \text{Replica 2 (Same Rack, Diff Node)} \to \text{Replica 3 (Different Rack)}$$

· 3. Step-by-Step Solved Examples

Example 1.1: Word Count in Python MapReduce Paradigm

- **Difficulty / Target Companies:** `Easy | Amazon, TCS Digital`
- **Problem Statement:** Implement Word Count using Map and Reduce functions.

Step-by-Step Walkthrough & Logic:

- Step 1: Mapper splits line into words, emitting `(word, 1)` for each.
- Step 2: Framework groups values by word key: `(word, [1, 1, 1])`.
- Step 3: Reducer sums list of ones, emitting `(word, total_count)`.

Complete Solved Code / Calculation:

```
def mapper(line: str):
    for word in line.strip().split():
        yield (word.lower(), 1)

def reducer(word: str, counts: list):
    yield (word, sum(counts))
```

- **Line-by-Line Breakdown & Complexity Analysis:** Illustrates functional decomposition of distributed counting across thousands of machines.

· 4. Practice Questions with Answers & Explanations

Practice Question 1.1

Question: What is the primary function of the HDFS NameNode?

Options:

- A) Executes user Java code
- B) Stores and manages all filesystem metadata, directory trees, and block location mappings in RAM
- C) Replaces hard drives
- D) Encrypts network cables

Answer: **B) Stores and manages all filesystem metadata, directory trees, and block location mappings in RAM** **Detailed Solution & Reasoning:** The NameNode maintains the in-memory metadata tree; if NameNode fails without High Availability, HDFS becomes inaccessible.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 1.1 (Amazon)

- **Interview Question:** _Why is NameNode memory (RAM) often the primary scalability bottleneck of large HDFS clusters?_
- **Model High-Scoring Answer:** Every file, directory, and block in HDFS consumes approximately 150 bytes of memory in the NameNode's RAM. If an application generates millions of small files (e.g., 1KB each), the NameNode runs out of RAM long before the DataNodes run out of disk storage ('Small Files Problem'). To mitigate this, data engineers compact small files into large columnar formats (Parquet/ORC) or sequence files.
- **Interviewer Follow-Up & Pro Tip:** ? Cite the HDFS Small Files Problem.

CHAPTER 2

Apache Spark Architecture & Distributed Processing

Key Principle / Exam Takeaway: Master Spark In-Memory computing, Driver vs Executor, RDD lineage, Catalyst Optimizer, and Narrow vs Wide transformations (Shuffles).

Core Curriculum Topics Covered in this Chapter:

- Spark vs MapReduce: In-memory DAG execution vs disk-spilling MapReduce
- Cluster architecture: Driver Program, SparkContext/SparkSession, Cluster Manager, Executors
- Resilient Distributed Datasets (RDDs): Immutable, partitioned collections with lazy evaluation
- Transformations vs Actions: Narrow (map, filter) vs Wide dependencies (groupByKey, join)
- DataFrames, Spark SQL, and Catalyst Query Optimizer (Logical -> Optimized -> Physical Plan)

· 1. Core Concepts & In-Depth Explanation

Apache Spark is a unified analytics engine providing 100x faster in-memory processing than MapReduce.

Core Spark Engine:

- **Driver:** Runs the `main()` method, constructs the Directed Acyclic Graph (DAG) of stages, and schedules tasks.
- **Executors:** Worker processes running on cluster nodes that execute tasks and cache data in RAM.
- **Transformations (Lazy):** Create a new RDD/DataFrame from an existing one (e.g., `filter()`, `map()`). Execution is deferred until an Action is called.
- **Actions (Eager):** Trigger job execution and return results to the Driver or write to storage (e.g., `count()`, `collect()`, `saveAsParquet()`).

Narrow vs Wide Dependencies:

- **Narrow Transformation:** Each partition of the parent RDD is used by at most one partition of the child RDD (e.g., `map()`, `filter()`). Runs locally in-memory without network transfers.
- **Wide Transformation:** Multiple child partitions depend on data across all parent partitions (e.g., `groupByKey()`, `join()`). Requires a **Shuffle** (spilling to disk and transferring data across the network), representing the primary performance bottleneck in Spark jobs.

· 2. Key Formulas, Rules & Complexity Classes

1. **Transformation Classification:** $$\text{Narrow (Zero Shuffle): } map(), filter(), union() \implies O(1) \text{ network transfer}$$ $$\text{Wide (Shuffle Barrier): } groupByKey(), reduceByKey(), join() \implies O(N) \text{ network I/O}$$

· 3. Step-by-Step Solved Examples

Example 2.1: Optimizing Word Count using reduceByKey instead of groupByKey

- **Difficulty / Target Companies:** `Medium | Amazon, Google, Uber`
- **Problem Statement:** Why is `reduceByKey()` preferred over `groupByKey()` in Apache Spark? Show PySpark implementation.

Step-by-Step Walkthrough & Logic:

- Step 1: `groupByKey()` shuffles all raw records across the network before aggregating.
- Step 2: `reduceByKey()` performs Map-side partial aggregation locally before shuffling, reducing network payload by 90%+.
- Step 3: PySpark implementation.

Complete Solved Code / Calculation:

```
# Optimal PySpark Implementation using DataFrames & reduceByKey
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("OptimalWordCount").getOrCreate()
lines = spark.sparkContext.textFile("hdfs://data/large_text.txt")

# map-side combine via reduceByKey:
counts = lines.flatMap(lambda line: line.split()) \
              .map(lambda word: (word.lower(), 1)) \
              .reduceByKey(lambda a, b: a + b)

counts.saveAsTextFile("hdfs://data/output_counts")
```

- **Line-by-Line Breakdown & Complexity Analysis:** Map-side combiners in reduceByKey collapse local partitions into single counts prior to network transmission.

· 4. Practice Questions with Answers & Explanations

Practice Question 2.1

Question: What is an RDD Lineage Graph in Apache Spark?

Options:

- A) Network map
- B) An internal DAG record of all transformations used to build an RDD, enabling automatic recomputation if a node crashes
- C) Database schema
- D) Security log

Answer: **B) An internal DAG record of all transformations used to build an RDD, enabling automatic recomputation if a node crashes** **Detailed Solution & Reasoning:** Fault tolerance in Spark is achieved via lineage tracking rather than replicating data across nodes.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 2.1 (Amazon)

- **Interview Question:** _What is Data Skew in a Spark job and how do you resolve it?_

- **Model High-Scoring Answer:** Data Skew occurs when data partitions are unevenly distributed such that one partition contains vastly more records than others (e.g., grouping by a `country` column where 90% of users are from 'US'). The single executor processing the hot partition runs out of memory or takes hours while other executors sit idle (the 'Straggler Task'). Solutions: 1) **Salting:** Append random integer suffixes (e.g., `US_1`, `US_2`) to the skew key to distribute it across multiple partitions, perform partial aggregation, and strip the salt. 2) **Broadcast Join:** If joining with a small dimension table, use `broadcast(small_df)` to eliminate shuffles entirely.

- **Interviewer Follow-Up & Pro Tip:** ? Cite Salting and Broadcast Joins as production remedies.

CHAPTER 3

Data Warehousing vs Data Lakes vs Lakehouses

Key Principle / Exam Takeaway: Master OLTP vs OLAP, Star vs Snowflake schema, Columnar Parquet/ORC storage, Data Lake zones, and Delta Lake ACID transactions.

Core Curriculum Topics Covered in this Chapter:

- OLTP (Row-oriented, normalized, high-write transactions) vs OLAP (Column-oriented, denormalized, analytical queries)
- Dimensional Modeling: Fact tables (metrics) and Dimension tables (context)
- Star Schema vs Snowflake Schema (normalized dimensions)
- Columnar file formats: Apache Parquet vs ORC (Snappy compression, dictionary encoding, predicate pushdown)
- The Lakehouse Paradigm: Delta Lake, Apache Iceberg, and Apache Hudi ACID layers

· 1. Core Concepts & In-Depth Explanation

Modern analytics architectures manage analytical data at scale:

- **Data Warehouse (OLAP - Snowflake, Amazon Redshift, BigQuery):** Stores structured, curated data in dimensional schemas for business intelligence.
- **Data Lake (Amazon S3, ADLS):** Stores raw, unstructured, semi-structured, and structured data cheaply at petabyte scale. Often suffered from lack of transactions ('Data Swamp').
- **Data Lakehouse (Delta Lake, Apache Iceberg):** Combines the cost efficiency of object storage (S3) with the ACID transaction guarantees, schema enforcement, time-travel, and indexing of relational warehouses.

Row vs Columnar Storage:

- **Row-Oriented (Postgres, CSV):** Stores full tuples contiguously. Optimal for OLTP (`SELECT * WHERE id = 101`).
- **Column-Oriented (Parquet, ORC):** Stores values of the same column contiguously. Optimal for OLAP aggregations (`SELECT AVG(salary) FROM emp`). Achieves 70%+ compression via Dictionary and Run-Length Encoding and supports **Predicate Pushdown** (skipping entire column chunks using Min/Max row group metadata).

· 2. Key Formulas, Rules & Complexity Classes

1. **Columnar Compression Advantage:** $$\text{Storage}(Parquet) \approx 0.20 \times \text{Storage}(CSV) \quad \text{(5x storage and I/O reduction)}$$

· 3. Step-by-Step Solved Examples

Example 3.1: Designing a Star Schema for E-Commerce Orders

- **Difficulty / Target Companies:** `Easy | Amazon, TCS Digital`
- **Problem Statement:** Design a Star Schema for analyzing retail sales across customers, products, stores, and time.

Step-by-Step Walkthrough & Logic:

- Step 1: Identify central business event -> `Fact_Sales`.

- Step 2: Fact table stores numeric additive metrics (quantity_sold, unit_price, discount, total_amount) and foreign keys.
- Step 3: Create surrounding Dimension tables: `Dim_Customer`, `Dim_Product`, `Dim_Store`, `Dim_Date`.

Complete Solved Code / Calculation:

```
-- Star Schema Fact Table:
CREATE TABLE Fact_Sales (
    sales_id BIGINT PRIMARY KEY,
    date_id INT REFERENCES Dim_Date(date_id),
    customer_id INT REFERENCES Dim_Customer(customer_id),
    product_id INT REFERENCES Dim_Product(product_id),
    store_id INT REFERENCES Dim_Store(store_id),
    units_sold INT NOT NULL,
    revenue DECIMAL(12,2) NOT NULL
);
```

- **Line-by-Line Breakdown & Complexity Analysis:** Denormalized dimensions allow high-speed joins with the central fact table in OLAP engines.

· 4. Practice Questions with Answers & Explanations

Practice Question 3.1

Question: What is Predicate Pushdown in Apache Parquet?

Options:

- A) Converting files to JSON
- B) Query engines inspect min/max metadata statistics stored in Parquet footers to skip reading entire row groups of irrelevant data
- C) Deleting rows
- D) Encrypting headers

Answer: **B) Query engines inspect min/max metadata statistics stored in Parquet footers to skip reading entire row groups of irrelevant data** **Detailed Solution & Reasoning:** If a query filters `WHERE year = 2026` and a Parquet row group's max year is 2024, the engine skips reading the entire block from disk.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 3.1 (Amazon)

- **Interview Question:** _What are Slowly Changing Dimensions (SCD Type 1 vs Type 2) and how are historical changes tracked in data warehouses?_
- **Model High-Scoring Answer:** SCD handles changes to dimension attributes over time. **SCD Type 1 (Overwrite):** Overwrites the existing attribute value with new data; zero historical record is preserved. **SCD Type 2 (Add New Row):** Preserves full historical auditability. When an attribute changes (e.g., a customer moves from Seattle to New York), a new record is inserted with the new address, using a surrogate key, a `start_date`, an `end_date`, and a boolean flag `is_current = True` (marking previous row `is_current = False`).
- **Interviewer Follow-Up & Pro Tip:** ? Cite SCD Type 2 start/end dates and current flags.

CHAPTER 4

Distributed Messaging & Streaming Architecture

Key Principle / Exam Takeaway: Master Apache Kafka architecture, Topics, Partitions, Consumer Groups, Producer ACKs, and Event-Driven Stream Processing.

Core Curriculum Topics Covered in this Chapter:

- Messaging paradigms: Point-to-point queues (RabbitMQ) vs Distributed commit log (Apache Kafka)
- Kafka Architecture: Brokers, Topics, Partitions, Replicas (Leader/Follower), ZooKeeper / KRaft
- Consumer Groups: Partition assignment, rebalancing, and horizontal consumer scaling
- Delivery Semantics: At-most-once, At-least-once, Exactly-once processing (Idempotent Producers & Transactions)
- Kafka retention policies, log compaction, and stream processing with Kafka Streams / Flink

· 1. Core Concepts & In-Depth Explanation

Apache Kafka is an append-only distributed commit log designed for high-throughput, fault-tolerant real-time event streaming:

- **Topic & Partitions:** A Topic is a category of events. Topics are horizontally split into **Partitions** ordered append-only sequence of immutable records. Partitions provide parallelism: each partition can reside on a different broker.
- **Consumer Groups:** Multiple consumer instances reading from a topic. Kafka assigns each partition to exactly ONE consumer in a group, enabling load balancing. Adding more consumers than partitions leaves excess consumers idle!
- **Delivery Guarantees:**
- **At-least-once:** Messages never lost, duplicates possible (standard).
- **Exactly-once (EOS):** Enabled via Idempotent Producers (`enable.idempotence=true`) and transactional two-phase commit across partitions.

· 2. Key Formulas, Rules & Complexity Classes

1. **Consumer Group Parallelism Invariant:** $$\text{Max Active Consumers in Group} = \text{Number of Topic Partitions}$$ 2. **Producer ACKs Settings:** $$\text{acks=0 (Zero wait)}, \quad \text{acks=1 (Leader only)}, \quad \text{acks=all / -1 (Leader + All ISRs)}$$

· 3. Step-by-Step Solved Examples

Example 4.1: Configuring Idempotent Producer in Kafka (Python)

- **Difficulty / Target Companies:** `Easy | Amazon, Uber`
- **Problem Statement:** Configure a Kafka producer guaranteeing zero duplicate messages on network retries.

Step-by-Step Walkthrough & Logic:

- Step 1: Set `acks='all'` to require acknowledgment from all in-sync replicas.

- Step 2: Enable `enable_idempotence=True`.
- Step 3: Set `retries` to maximum.

Complete Solved Code / Calculation:

```
# Conceptual Python Producer Configuration:
producer_conf = {
    'bootstrap.servers': 'kafka1:9092,kafka2:9092',
    'acks': 'all',                  # Guarantees durability
    'enable.idempotence': True,     # Prevents duplicate writes on retry
    'max.in.flight.requests.per.connection': 5,
    'retries': 1000000
}
```

- **Line-by-Line Breakdown & Complexity Analysis:** Kafka assigns a unique Producer ID and monotonic sequence numbers to prevent duplicate commits upon retry.

· 4. Practice Questions with Answers & Explanations

Practice Question 4.1

Question: If a Kafka topic has 4 partitions and a consumer group has 6 consumer instances, how many consumers will sit idle?

Options:

- A) 0
- B) 2
- C) 4
- D) 1

Answer: **B) 2** **Detailed Solution & Reasoning:** Each partition is assigned to at most one consumer per consumer group, leaving 6 - 4 = 2 idle consumers.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 4.1 (Uber)

- **Interview Question:** _How does Apache Kafka achieve millions of messages per second throughput on standard commodity hardware?_
- **Model High-Scoring Answer:** Kafka leverages 4 key OS and hardware design principles: 1) **Sequential I/O:** Appending to sequential disk logs is as fast as RAM random access, avoiding random disk seeks. 2) **Zero-Copy Optimization:** Uses Linux `sendfile()` system call to transfer bytes directly from OS Page Cache to the network NIC socket buffer without copying into user-space JVM memory. 3) **Batching & Compression:** Batches messages together on producer and wire (Snappy/LZ4). 4) **Partition Parallelism:** Horizontal scaling across partition leaders.

- **Interviewer Follow-Up & Pro Tip:** ? Emphasize Linux Zero-Copy `sendfile()` and sequential disk writes.

CHAPTER 5

ETL / ELT Pipeline Architecture & Data Orchestration

Key Principle / Exam Takeaway: Master ETL vs ELT, Apache Airflow DAGs, Task dependencies, Backfilling, and Idempotent data engineering pipelines.

Core Curriculum Topics Covered in this Chapter:

- ETL (Extract, Transform, Load) vs modern cloud ELT (Extract, Load, Transform in warehouse)
- Apache Airflow architecture: Webserver, Scheduler, Metadata Database, Executor (Celery/Kubernetes)
- Directed Acyclic Graph (DAG) design, operators (PythonOperator, BashOperator), and task sensors
- Idempotency in Data Engineering: Safe re-runs, upserts (MERGE), and partition overwrites
- Backfilling historical data and catchup parameter management

· 1. Core Concepts & In-Depth Explanation

Data pipelines move and process information between source databases and analytical targets:

- **ETL:** Legacy model where compute-heavy transformations occurred in dedicated middle-tier ETL servers before loading into databases.
- **ELT:** Modern paradigm enabled by cloud data warehouses (Snowflake, BigQuery). Raw data is loaded directly into the warehouse, and transformations are performed in-warehouse using high-performance SQL.

Data Orchestration with Apache Airflow: Airflow structures pipelines as **Directed Acyclic Graphs (DAGs)**. Tasks represent individual units of work (Nodes), while dependencies (`task_a >> task_b`) define execution sequence (Edges).

Idempotency (The Golden Rule): A pipeline is **Idempotent** if running it multiple times with the identical input parameters produces the identical end state without creating duplicate records or data corruption. Achieved via: 1. Deleting target partitions before loading (`INSERT OVERWRITE`). 2. Using atomic `MERGE` (upsert) statements based on primary keys.

· 2. Key Formulas, Rules & Complexity Classes

1. **Idempotency Equation:** $$f(f(x)) = f(x) \quad \text{(Re-running a failed batch produces identical correct state)}$$

· 3. Step-by-Step Solved Examples

Example 5.1: Designing an Idempotent Apache Airflow DAG

- **Difficulty / Target Companies:** `Medium | Amazon, TCS Digital`
- **Problem Statement:** Write an Airflow DAG with sequential extract, transform, and load tasks.

Step-by-Step Walkthrough & Logic:

- Step 1: Define DAG context with schedule interval and start date.
- Step 2: Define tasks using `PythonOperator`.
- Step 3: Establish dependency chain using bitshift operators: `extract >> transform >> load`.

Complete Solved Code / Calculation:

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime, timedelta

default_args = {
    'owner': 'data_eng',
    'retries': 2,
    'retry_delay': timedelta(minutes=5)
}

def extract_data(): print("Extracted")
def transform_data(): print("Transformed")
def load_data(): print("Loaded")

with DAG('daily_sales_pipeline',
         default_args=default_args,
         start_date=datetime(2026, 1, 1),
         schedule_interval='@daily',
         catchup=False) as dag:

    t1 = PythonOperator(task_id='extract', python_callable=extract_data)
    t2 = PythonOperator(task_id='transform', python_callable=transform_data)
    t3 = PythonOperator(task_id='load', python_callable=load_data)

    t1 >> t2 >> t3 # Dependency chain
```

- **Line-by-Line Breakdown & Complexity Analysis:** Guarantees deterministic execution sequence and automatic retries on network failures.

· 4. Practice Questions with Answers & Explanations

Practice Question 5.1

Question: Why is `catchup=False` frequently recommended in Apache Airflow DAG definitions?

Options:

- A) Speeds up scheduler
- B) Prevents Airflow from triggering dozens of backfill DAG runs between start_date and current time upon first deployment
- C) Disables tasks
- D) Deletes logs

Answer: **B) Prevents Airflow from triggering dozens of backfill DAG runs between start_date and current time upon first deployment** **Detailed Solution & Reasoning:** If catchup is true, Airflow attempts to execute every historical missed schedule interval back to start_date, which can overwhelm databases.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 5.1 (Amazon)

- **Interview Question:** _What is the difference between an Airflow Operator, a Sensor, and a Hook?_

- **Model High-Scoring Answer:** An **Operator** defines a single atomic unit of work (e.g., executing Python code, running a Bash command, or submitting a Spark job). A **Sensor** is a specialized operator that pauses and waits for an external event to occur (e.g., waiting for an S3 file to arrive or a table partition to populate) before unblocking downstream tasks. A **Hook** is a reusable low-level connection wrapper handling authentication, pooling, and API calls to external systems (e.g., `PostgresHook`, `S3Hook`), serving as the foundation upon which operators are built.

- **Interviewer Follow-Up & Pro Tip:** ? Differentiate work (Operator), waiting (Sensor), and connection (Hook).

CHAPTER 6

NoSQL Databases: Document, Key-Value, Columnar & Graph

Key Principle / Exam Takeaway: Master the 4 NoSQL archetypes (Document, Key-Value, Wide-Column, Graph), Cassandra partition keys, MongoDB aggregation pipelines, and Neo4j Cypher.

Core Curriculum Topics Covered in this Chapter:

- NoSQL drivers: High write throughput, horizontal scalability, schema flexibility, non-relational structures
- Document Stores (MongoDB): BSON documents, collections, indexes, and aggregation pipelines
- Key-Value Stores (Redis, DynamoDB): O(1) in-memory lookups, TTL caching, single-digit millisecond latency
- Wide-Column Stores (Apache Cassandra): LSM-Trees, Partition keys vs Clustering keys, tunable consistency
- Graph Databases (Neo4j): Property graph model (Nodes, Edges, Properties) and index-free adjacency

· 1. Core Concepts & In-Depth Explanation

NoSQL databases address requirements where relational schemas become bottlenecks: 1. **Document Stores (MongoDB):** Stores hierarchical JSON/BSON documents. Ideal for catalogs, user profiles, and polymorphic records. 2. **Key-Value Stores (Redis):** Fast memory-resident key-value access. Ideal for session caches, real-time leaderboards, and pub/sub buffers. 3. **Wide-Column Stores (Cassandra):** Masterless distributed architecture based on Google Bigtable and Amazon Dynamo. Partition Key determines which node stores the row; Clustering Columns sort rows physically within the partition on disk. 4. **Graph Databases (Neo4j):** Stores interconnected networks. Features **Index-Free Adjacency**: each node holds direct physical memory pointers to its neighboring nodes, allowing complex relationship traversals in $O(1)$ time per hop regardless of total database size.

· 2. Key Formulas, Rules & Complexity Classes

1. **Cassandra Primary Key Syntax:** $$\text{PRIMARY KEY } ((K_{\text{partition1}}, K_{\text{partition2}}), C_{\text{cluster1}}, C_{\text{cluster2}})$$ $$\text{Partition Key (Distribution node)} + \text{Clustering Keys (In-node sorting order)}$$

· 3. Step-by-Step Solved Examples

Example 6.1: Cassandra Data Modeling for User Activity Stream

- **Difficulty / Target Companies:** `Medium | Amazon, Uber, Netflix`
- **Problem Statement:** Model a Cassandra table storing user activities such that all events for a user are clustered and sorted chronologically descending.

Step-by-Step Walkthrough & Logic:

- Step 1: Partition key must be `user_id` so all of a user's data resides on the same node.
- Step 2: Clustering key must be `event_timestamp` with `CLUSTERING ORDER BY (event_timestamp DESC)`.

- Step 3: Write CQL schema.

Complete Solved Code / Calculation:

```
CREATE TABLE user_activities (
    user_id UUID,
    event_id UUID,
    event_timestamp TIMESTAMP,
    activity_type TEXT,
    payload TEXT,
    PRIMARY KEY ((user_id), event_timestamp, event_id)
) WITH CLUSTERING ORDER BY (event_timestamp DESC);
```

- **Line-by-Line Breakdown & Complexity Analysis:** Allows fast contiguous sequential disk reads for `WHERE user_id = ? ORDER BY event_timestamp DESC` without sorting at query time.

· 4. Practice Questions with Answers & Explanations

Practice Question 6.1

Question: What is Index-Free Adjacency in Graph Databases like Neo4j?

Options:

- A) Database has no indexes
- B) Nodes maintain direct physical memory pointers to neighboring nodes, avoiding index lookups during relationship traversals
- C) Faster disk formatting
- D) Uses B+ trees only

Answer: **B) Nodes maintain direct physical memory pointers to neighboring nodes, avoiding index lookups during relationship traversals** **Detailed Solution & Reasoning:** Traversing relationships involves following direct pointers, delivering $O(1)$ hop performance.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 6.1 (Amazon)

- **Interview Question:** _How does Apache Cassandra handle writes using Memtables, CommitLogs, and SSTables?_
- **Model High-Scoring Answer:** Cassandra is optimized for blazing write performance using an LSM-Tree (Log-Structured Merge-Tree) architecture: 1) Every write is immediately appended to an on-disk **CommitLog** for crash durability. 2) The write is simultaneously inserted into an in-memory sorted buffer called a **Memtable**. 3) Once the write hits the Memtable, the client receives acknowledgment (making writes faster than reads!). 4) When the Memtable fills up, it flushes sequentially to disk as an immutable **SSTable** (Sorted String Table). 5) Background **Compaction** merges multiple SSTables to reclaim deleted space and reconcile updates.
- **Interviewer Follow-Up & Pro Tip:** ? Trace: CommitLog (disk) + Memtable (RAM) -> SSTable (immutable disk) -> Compaction.

CHAPTER 7

Data Modeling, Partitioning & Sharding at Scale

Key Principle / Exam Takeaway: Master horizontal sharding, partition keys, hotspot prevention, range vs hash partitioning, and distributed transactions.

Core Curriculum Topics Covered in this Chapter:

- Horizontal Sharding vs Vertical Partitioning
- Partition Key selection criteria: Cardinality, distribution uniformity, query patterns
- Hotspot prevention: Celebrity problem / heavy key mitigation
- Range Partitioning vs Hash Partitioning vs List Partitioning
- Distributed joins and cross-shard transaction challenges

· 1. Core Concepts & In-Depth Explanation

Scaling massive data storage requires partitioning datasets across independent database instances:

- **Vertical Partitioning:** Splitting columns across tables (e.g., moving infrequently accessed BLOBs/profile bios to a separate table).
- **Horizontal Sharding:** Splitting rows across distinct physical servers, where each server (shard) contains identical schema definitions.
- **Partition Key Selection:** The most critical decision in distributed engineering:
- **High Cardinality:** The key must have millions of distinct values (e.g., `user_id` or `order_id`). Low cardinality keys (e.g., `country`) cause uneven shards.
- **Uniform Distribution:** Avoid keys that produce skewed traffic ('Celebrity Problem').
- **Query Alignment:** The partition key should match the `WHERE` clauses of 80%+ of incoming queries, routing queries directly to a single shard (**Single-Shard Query**) rather than querying all shards (**Scatter-Gather Query**).

· 2. Key Formulas, Rules & Complexity Classes

1. **Shard Routing:** $$\text{ShardID} = \text{hash}(\text{PartitionKey}) \pmod{\text{NumShards}}$$ 2. **Scatter-Gather Latency:** $$\text{Latency}(\text{Scatter-Gather}) = \max_{i=1}^N(\text{Latency}(\text{Shard}_i)) \implies \text{Vulnerable to slowest straggler}$$

· 3. Step-by-Step Solved Examples

Example 7.1: Mitigating the Celebrity Problem in Partitioned Systems

- **Difficulty / Target Companies:** `Medium | Amazon, Twitter/X`
- **Problem Statement:** A social network partitions user timelines by `user_id`. A celebrity with 50M followers creates massive write amplification when posting a tweet. How is this resolved?

Step-by-Step Walkthrough & Logic:

- Step 1: For ordinary users, use **Fan-out on Write**: push tweet to all followers' timeline shards.
- Step 2: For celebrity users (high follower count), use **Fan-out on Read**: do NOT push to 50M mailboxes on write.
- Step 3: When a follower loads their feed, their client merges the normal mailbox with tweets fetched dynamically from the celebrity's timeline at read time.

Complete Solved Code / Calculation:

```
// Hybrid Fan-out Strategy:
// Ordinary User Post: Push to followers (Fan-out on Write)
// Celebrity Post (> 100K followers): Lazy pull at read time (Fan-out on Read)
```

- **Line-by-Line Breakdown & Complexity Analysis:** Hybrid push-pull architectures resolve hotspot write bottlenecks at massive scale.

· 4. Practice Questions with Answers & Explanations

Practice Question 7.1

Question: What is a Scatter-Gather query in a sharded database?

Options:

- A) A query that fails
- B) A query lacking a partition key in its WHERE clause, forcing the coordinator to broadcast the query to every single shard and merge results
- C) Faster query
- D) In-memory query

Answer: **B) A query lacking a partition key in its WHERE clause, forcing the coordinator to broadcast the query to every single shard and merge results** **Detailed Solution & Reasoning:** Without a partition key, the router cannot identify the target shard, requiring an expensive cluster-wide broadcast.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 7.1 (Amazon)

- **Interview Question:** _What is the Two-Phase Commit (2PC) protocol for distributed transactions across multiple shards?_
- **Model High-Scoring Answer:** 2PC coordinates atomic distributed transactions across multiple shard nodes: 1) **Prepare Phase:** The Coordinator node sends a `PREPARE` message to all participant shards. Each participant executes the transaction locally, writes redo/undo logs, acquires locks, and replies `VOTE_COMMIT` or `VOTE_ABORT`. 2) **Commit Phase:** If all participants vote commit, the coordinator writes a commit record and broadcasts `GLOBAL_COMMIT`. If even one participant votes abort (or times out), the coordinator broadcasts `GLOBAL_ABORT`, and all nodes rollback. Weakness: 2PC is a blocking protocol; if the coordinator crashes during phase 2, participants remain locked.
- **Interviewer Follow-Up & Pro Tip:** ? Cite the blocking vulnerability of coordinator failure.

CHAPTER 8

Solved Data Engineering Placement Case Studies & System Interview Dr

Key Principle / Exam Takeaway: Master end-to-end data architectures: Real-time fraud detection pipeline, Clickstream web analytics, and Batch vs Streaming Lambda/Kappa architectures.

Core Curriculum Topics Covered in this Chapter:

- Architecture Archetypes: Lambda Architecture (Batch + Speed layers) vs Kappa Architecture (Streaming log only)
- Case Study 1: Real-Time Fraud Detection Pipeline (Kafka, Flink, Redis)
- Case Study 2: High-Throughput Clickstream Analytics Engine (Kafka, Spark Streaming, Parquet, Snowflake)
- Data Quality, Schema Evolution with Avro and Confluent Schema Registry
- Comprehensive Data Engineering placement interview rubrics

· 1. Core Concepts & In-Depth Explanation

This capstone chapter synthesizes data engineering components into enterprise production architectures:

- **Lambda Architecture:** Combines a **Batch Layer** (Hadoop/Spark, immutable historical truth, slow) and a **Speed Layer** (Flink/Storm, low latency, approximate) serving queries through a unified view. Suffers from dual-codebase maintenance.
- **Kappa Architecture:** Eliminates dual codebases by processing ALL data?both real-time and historical replays?through a single stream processing engine (Kafka + Apache Flink).

Real-Time Fraud Detection Pipeline: 1. Ingestion: Financial transaction events streamed into Apache Kafka. 2. Stream Processing: Apache Flink computes sliding-window aggregations (e.g., 'number of transactions > $1000 in last 5 minutes'). 3. Feature Store: Redis stores user profile features and velocity limits for $O(1)$ lookups. 4. Evaluation: ML scoring model flags anomalies; alerts sent to fraud investigation queues.

· 2. Key Formulas, Rules & Complexity Classes

1. **Architecture Progression:** $$\text{Lambda Architecture (Batch + Speed dual codebases)} \to \text{Kappa Architecture (Single streaming log)}$$

· 3. Step-by-Step Solved Examples

Example 8.1: End-to-End Clickstream Ingestion Architecture Design

- **Difficulty / Target Companies:** `Hard | Amazon, Google, Netflix`
- **Problem Statement:** Design a pipeline capturing 50,000 click events per second from global web clients, providing dashboards with < 5 second latency and long-term analytical storage.

Step-by-Step Walkthrough & Logic:

- Step 1: Ingestion: Clients transmit JSON beacons to an API Gateway / Load Balancer, which writes directly into an Apache Kafka topic partitioned by `user_id`.

- Step 2: Real-Time Stream Layer: Apache Flink consumes Kafka topic, computing 1-minute tumbling window metrics (active page visitors, conversion rates), persisting counts to Redis for live dashboard visualization (< 2s latency).
- Step 3: Long-Term Batch Layer: Kafka Connect (or Spark Streaming) streams raw events in micro-batches to Amazon S3, partitioned by `year/month/day/hour` in Apache Parquet format.
- Step 4: Analytics: Snowflake / BigQuery queries S3 Parquet tables directly for deep business intelligence.

Complete Solved Code / Calculation:

```
// Data Flow Topology:
// Web Clients -> API Gateway -> Kafka -> Flink -> Redis (Live Dashboards)
//                                    -> S3 (Parquet) -> Snowflake (Deep BI)
```

- **Line-by-Line Breakdown & Complexity Analysis:** Decouples low-latency operational dashboarding from high-throughput historical analytical queries.

· 4. Practice Questions with Answers & Explanations

Practice Question 8.1

Question: What is the primary advantage of the Kappa Architecture over the Lambda Architecture?

Options:

- A) Cheaper hardware
- B) Maintains a single unified stream processing codebase for both real-time and historical reprocessing, eliminating dual-codebase bugs
- C) Eliminates databases
- D) Uses relational tables only

Answer: **B) Maintains a single unified stream processing codebase for both real-time and historical reprocessing, eliminating dual-codebase bugs** **Detailed Solution & Reasoning:** Kappa processes historical data simply by streaming past Kafka logs through the identical stream processing code.

· 5. Real Company Technical Interview Questions & Answers

Interview Question 8.1 (Amazon)

- **Interview Question:** _How do you handle schema evolution when producing and consuming events in Kafka to prevent breaking downstream pipelines?_
- **Model High-Scoring Answer:** Use a centralized **Schema Registry** (e.g., Confluent Schema Registry) paired with a binary serialization format like **Apache Avro** or **Protobuf**. Before producing a message, the producer checks the schema with the registry. Each message carries a lightweight 4-byte Schema ID in its header. Consumers look up the schema ID to deserialize the payload. Enforce **Backward Compatibility** rules: fields can be added if they have default values, and deleted fields must have been optional, ensuring older consumer applications can read newer event schemas without crashing.
- **Interviewer Follow-Up & Pro Tip:** ? Cite Schema Registry with Avro and backward compatibility rules.

· Campus Placement Certification & Final Exam Policy

To earn Placement-Ready Master status, students must complete all chapter practice questions, review all solved examples, and score at least 80% on the comprehensive subject assessment.